Concurrency & Distributed Systems

Week 2 — Day 1: Threads: Mental Model + Creation Patterns

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

Disruptive Question

Think

If I create two threads in Java...
Do they really run at the same time?

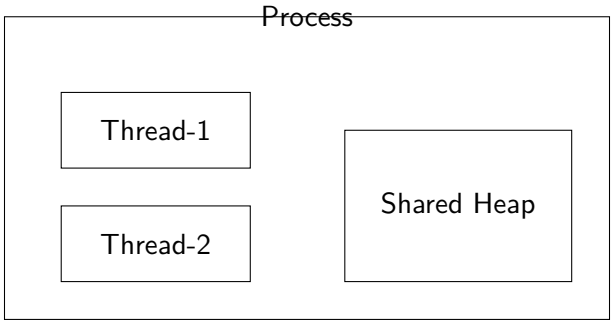
If I create 8 threads on a 4-core CPU...
What is actually happening?

Illusion vs Reality

$$\text{Illusion} \neq \text{Reality}$$

- Concurrency is not magic
- The OS decides execution
- Order is not guaranteed

What Is a Process?



Thread Memory Model

Each Thread Has

- Its own Stack
- Its own Program Counter
- Its own Local Variables

All Threads Share

- Heap (Objects)
- Static Variables
- Files / Sockets

Critical Precision

$$\text{Reference Location} \neq \text{Object Location}$$

- Local variable may be private
- Object it points to may be shared
- This is where race conditions begin

Who Schedules Threads?

Important

JVM creates threads.
OS schedules them.

$$\text{JVM} \neq \text{CPU Scheduler}$$

Execution order is **nondeterministic**.

Time Slicing (Reality)

T1 T2 T1 T2 T2 T1

Threads are:

- Preempted
- Interleaved
- Time-sliced

Interrupt Discipline

Never Swallow Interrupts

```
catch (InterruptedException e) {
        Thread.currentThread().interrupt();
}
```

- Interrupt = cancellation signal
- Swallowing creates zombie threads
- Professional Java respects interruption

Pattern 1 — Extending Thread

```
class MyThread extends Thread {
        public void run() {
                System.out.println(getName());
        }
}

MyThread t = new MyThread();
t.start();
```

- Task tightly coupled to Thread
- Single inheritance limitation

Pattern 2 — Runnable (Preferred)

```
class MyTask implements Runnable {
        public void run() {
                System.out.println(
                Thread.currentThread().getName()
                );
        }
}

Thread t = new Thread(new MyTask());
t.start();
```

$$\text{Task} \neq \text{Thread}$$

Pattern 3 — Lambda

```
Thread t = new Thread(() -> {
        System.out.println(
        Thread.currentThread().getName()
        );
}, "Lambda-Worker");

t.start();
```

Runnable = Functional Interface

Critical Mistake: run() vs start()

```
Thread t = new Thread(() ->
System.out.println(
Thread.currentThread().getName()
)
);

t.run(); // What happens?
```

Critical Mistake: run() vs start()

```
Thread t = new Thread(() ->
System.out.println(
Thread.currentThread().getName()
)
);

t.run(); // What happens?
```

Prints: **main**

Correct Way

```
t.start();
```

$$\text{run}() = \text{synchronous}$$
$$\text{start}() = \text{asynchronous}$$

start() creates a new call stack.

Interleaving Demonstration

```
Thread t1 = new Thread(() -> {
       for (int i = 0; i < 5; i++)
       System.out.println("T1-" + i);
});

Thread t2 = new Thread(() -> {
       for (int i = 0; i < 5; i++)
       System.out.println("T2-" + i);
});

t1.start();
t2.start();
```

Execution order is unpredictable.

Sleep Myth

```
Thread.sleep(1);
```

- Sleep does NOT guarantee order
- Sleep does NOT enforce fairness
- Sleep does NOT release locks

Block 1 Summary

- Thread = schedulable execution unit
- Stack = private
- Heap = shared
- OS schedules
- run() $\neq$ start()
- Order is not guaranteed
- Interleaving is normal

Today you lost control over time.
Tomorrow you regain control over access.